

Task 1 — Python Automation: Smart File Organizer

Objective

Develop a Python-based automation system that simplifies file management by automatically organizing files into categorized folders, creating backups, renaming files, detecting duplicates, cleaning empty directories, and generating detailed reports. The project reduces manual effort and improves file organization efficiency.

Tools & Technologies

- Python 3.x
 - Flask
 - OS Module
 - Shutil
 - Hashlib
 - Zipfile
 - Logging
 - HTML5
 - CSS3
 - Command Line / Terminal
-

How It Works

1. Folder Selection

A web-based interface allows the user to input the target folder path for organization.

2. Automatic Backup

Before any modifications, the system creates a ZIP backup of the selected folder to ensure data safety.

3. File Organization

Files are automatically sorted into categorized folders based on their extensions.

Example Categories:

- **Images:** .jpg, .jpeg, .png, .gif
- **Documents:** .pdf, .docx, .txt
- **Audio:** .mp3, .wav
- **Videos:** .mp4, .avi

- **Programs:** .py, .java, .cpp

4. Bulk File Renaming

Users can specify a custom prefix, and files are renamed automatically.

Example:

- Before: image1.jpg
- After: College_image1.jpg

5. Empty Folder Cleanup

After processing, the system identifies and removes unnecessary empty folders.

6. Duplicate File Detection

Duplicate files are detected using file hashing techniques and reported to the user.

7. Report Generation

A detailed report is generated containing:

- Files moved
- Files renamed
- Empty folders removed
- Duplicate files found
- Backup status
- Execution timestamp

8. Logging

All actions are recorded in a log file for monitoring and troubleshooting.

Key Features

- Automatic File Organization
- Backup Creation
- Bulk File Renaming
- Duplicate File Detection
- Empty Folder Cleanup
- Activity Logging
- Report Generation
- User-Friendly Web Interface
- Exception Handling
- File Safety Protection

Key Code Snippet

```
def organize_files(folder):
    categories = {
        "Images": [".jpg", ".jpeg", ".png"],
        "Documents": [".pdf", ".docx", ".txt"],
        "Audio": [".mp3"],
        "Videos": [".mp4"]
    }

    for file in os.listdir(folder):
        file_path = os.path.join(folder, file)
        if os.path.isfile(file_path):
            ext = os.path.splitext(file)[1].lower()
            for category, extensions in categories.items():
                if ext in extensions:
                    target_folder = os.path.join(folder, category)
                    os.makedirs(target_folder, exist_ok=True)
                    shutil.move(file_path, os.path.join(target_folder, file))
                    logging.info(f"Moved {file} -> {category}")
                    break
```

Sample Input

Folder Path:

C:\Users\Username\Desktop\TestFolder

Rename Prefix:

task1

Folder Contents Before Execution:

```
TestFolder/
|
|— image1.jpg
|— image2.png
|— report.pdf
|— song.mp3
|— video.mp4
```

Sample Output

Backup Created : backup_20260618.zip

Files Moved : 5

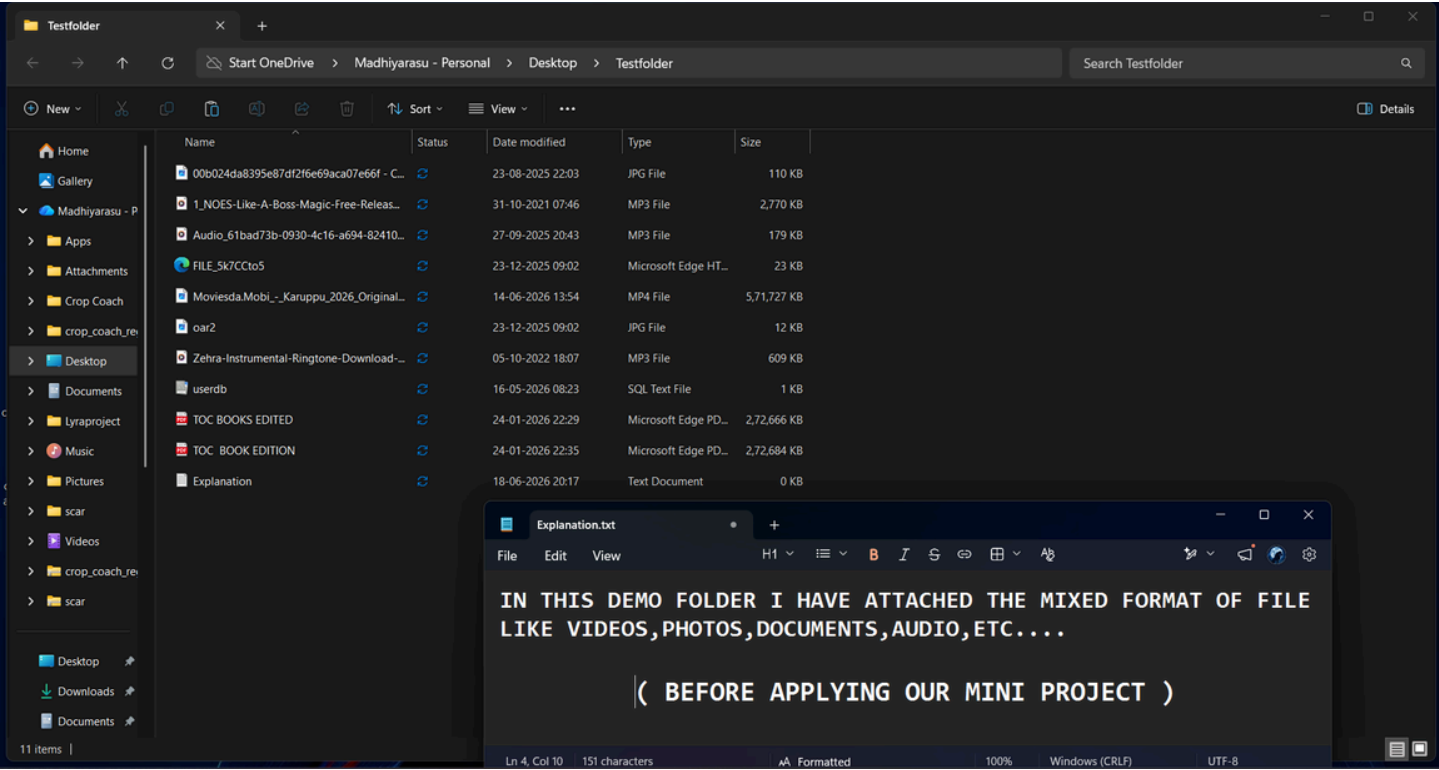
Files Renamed : 5

Empty Folders Removed : 1

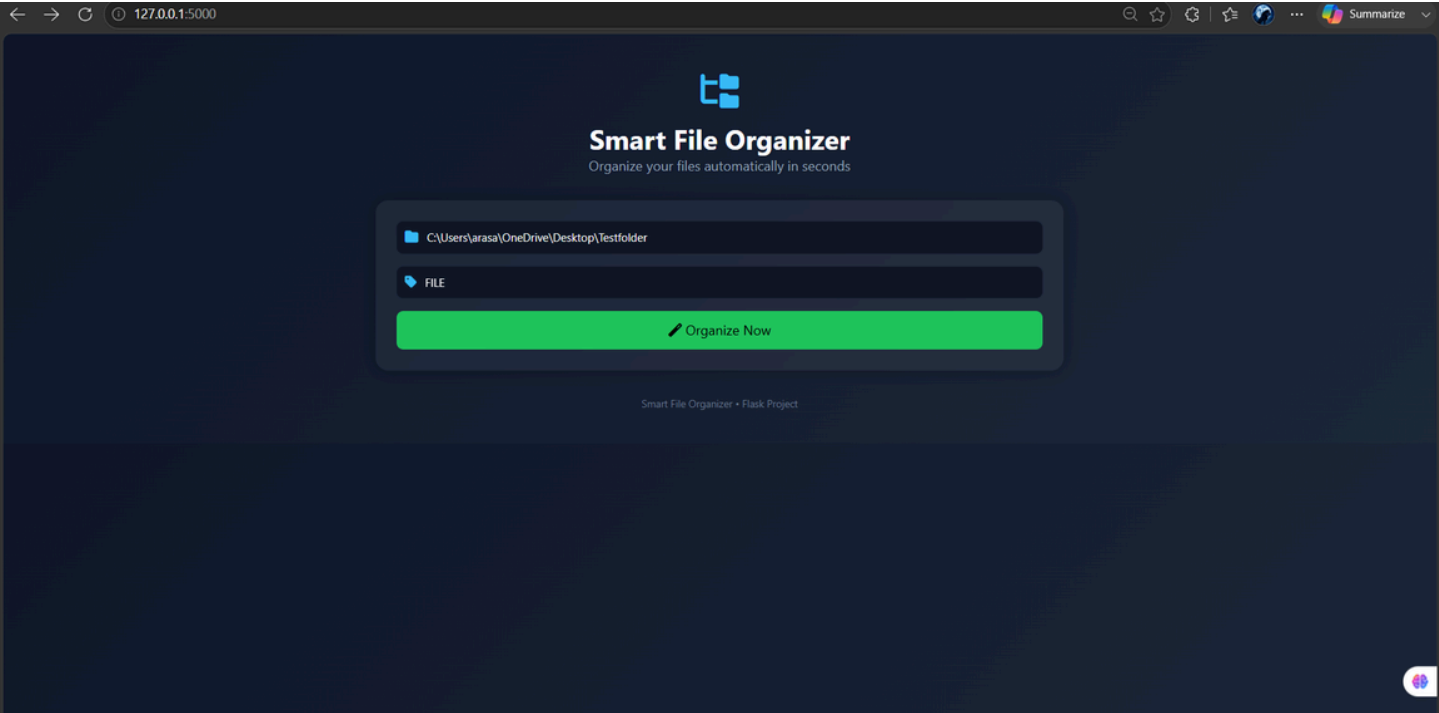
Duplicates Found : 0
Report Generated Successfully

Screenshots

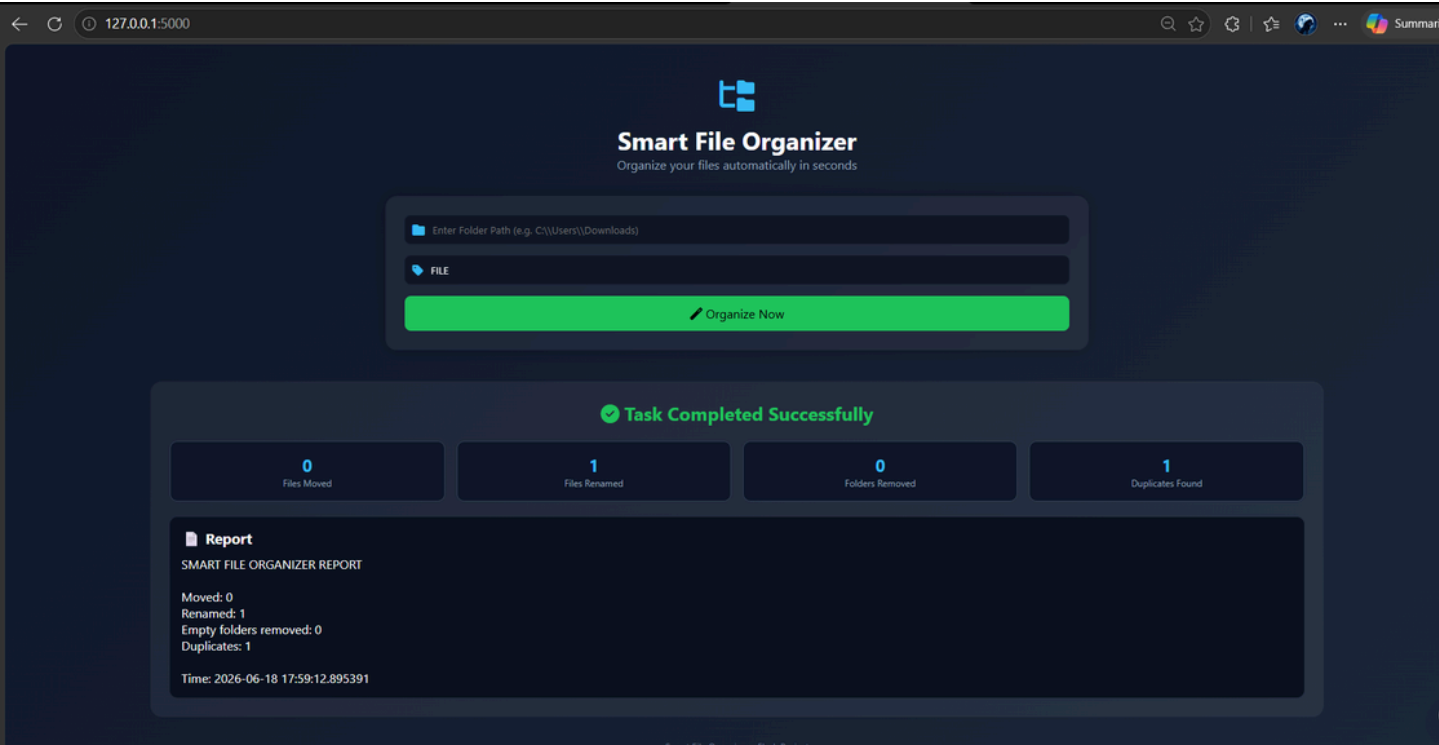
Screenshot 1 — Folder Structure Before Organization



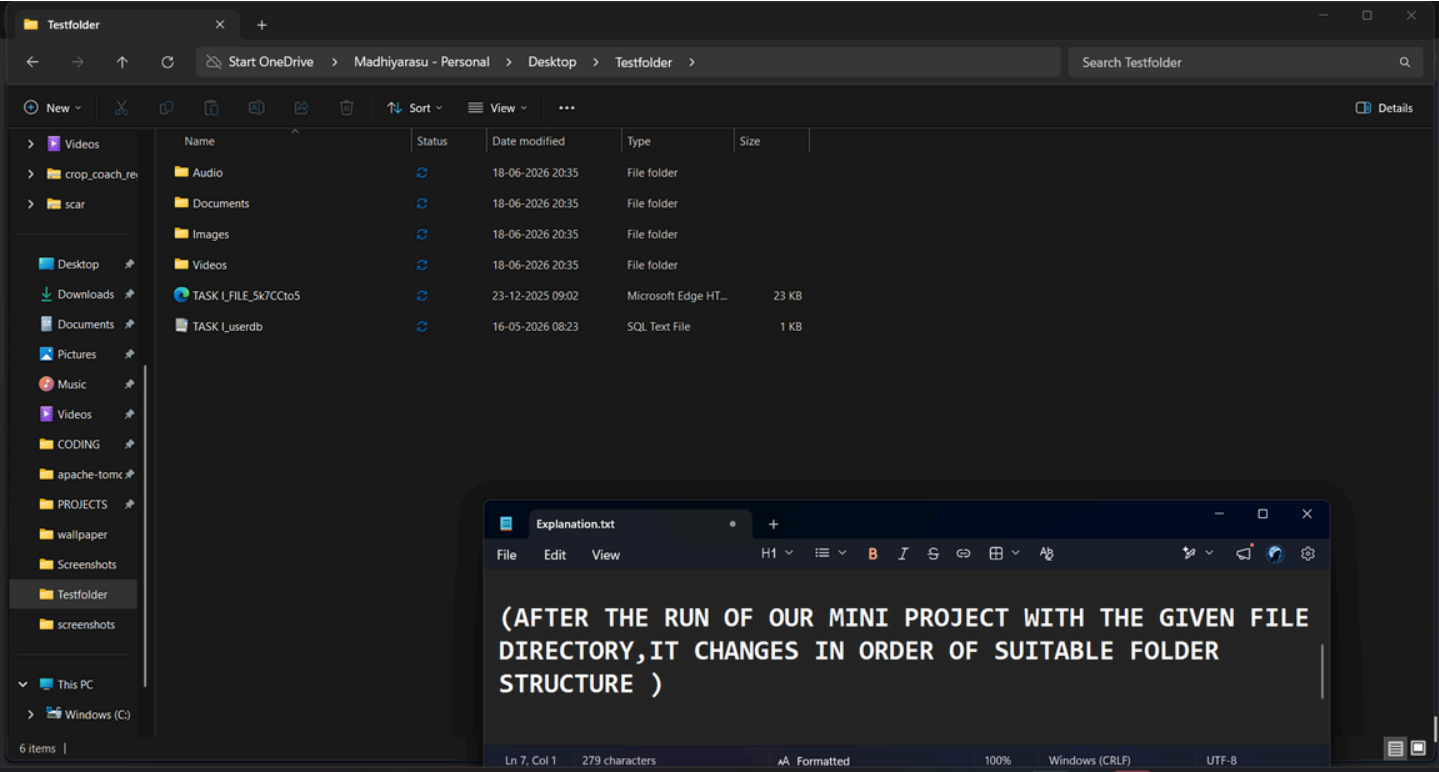
Screenshot 2 — Web Interface Before Execution



Screenshot 3— Automation Result Dashboard



Screenshot 4 — Generated Folder Structure After Organization



Project Structure

Task1-Python-Automation/

```
|
|
|— app.py
|— organizer.log
|— report.txt
|— requirements.txt
|
|— modules/
|   |— organizer.py
|
|— templates/
|   |— index.html
|
|— static/
|   |— style.css
|
|— backups/
```

Learning Outcomes

- Gained hands-on experience in **Python Automation**
 - Learned advanced **File Handling Operations**
 - Implemented **Exception Handling** and **Logging**
 - Developed a **Flask Web Application**
 - Understood **File Organization Techniques**
 - Applied **Duplicate Detection Algorithms**
 - Practiced **Backup and Recovery Mechanisms**
-

Submission Links

GitHub Repository:

[<https://github.com/mathiarasan-art01/InternSpark-Internship>]

Hosted Application:

Will be updated after deployment

Notes

Run the application using:

```
python app.py
```

Access the web interface at:

```
http://127.0.0.1:5000
```

The application automatically organizes files, creates backups, renames files, detects duplicates, removes empty folders, generates reports, and logs all operations. All source code, documentation, screenshots, and reports are included in the GitHub repository submitted for evaluation.
